

Developer Onboarding Validation

08/13/2026 16:20:23

<https://chatgpt.com/g/g-p-6a71ed6ea4d0819190118e1b9a94bea9-sport-analytics-project/c/6a7dc3c6-eae8-83ea-b076-4a8a292f8181>

Validate developer onboarding and motivate the technology stack #1...

Thinking 🌐 Remembering ● Reviewed COMS3011A methodologies, Git...

Sport-Analytics-Tool (2)(1).zip 1. can we do a pr and then in that ask...

Thinking 🌐 Inspecting ZIP archive contents ● Inspected archive conten...

yes, i think u correct, where does wrangler come in tho?

Thinking 🌐 Inspecting Project Deployment Files ● Inspected deployme...

do it

Thinking 🌐 Inspecting Project Repository Structure ● Clarifying...

i need to prettier alot of files, give me command (my machine needs...

Since you haven't committed yet, I'd format only the files Prettier flagg...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> python -m...

Thinking 🌐 Reviewing Patch Changes and Test Coverage ● Assessing...

vite v5.4.21 building for production... ✓ 136 modules transformed...

Thinking 🌐 Inspecting Patch Changes for Bundle Warning ● Inspected...

(use "git add <file>..." to update what will be committed) (use "git...

Thinking 🌐 Inspecting Repository Gitignore and Generated Site Files ●...

give commit messages again with Ai decleration in them, ie Assisted by

Yep - use the AI declaration on each commit for this issue since ChatGP...

when should i upload this transcript? and update the line in my ai...

Thinking ● Timing transcript export I'm recommending export after...

give me the line to add to my csv and what to call this transcript and gi...

Thinking 🌐 Inspecting AI register and methodology files ● Preparing P...

give step by step of what teammate needs to follow inside the pr, so re...

Thinking ● Drafting PR instructions I'm revising the PR description to...

noo, cant we refer them to a doc or somethoing instead of putting it all ...

Thinking 🌐 Inspecting Verification Document Content ● Inspected...



Validate developer onboarding and motivate the technology stack #12

****Open****

opened last week by [Shayna]
(<https://sdp.ms.wits.ac.za/Shayna>) · 0 comments

[****Shayna****]
(<https://sdp.ms.wits.ac.za/Shayna>) commented [last week](<https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/issues/12#issue-14>)

****Member****

Description

Test the complete developer setup process from a clean clone and improve the onboarding and technology-stack documentation.

The final documentation must allow a team member to install dependencies, configure the environment, run the frontend and backend, execute the repository checks and understand why each major technology was selected.

Motivation

The project must be maintainable by the full team rather than only by the person who created the initial scaffold.

Clear onboarding instructions reduce setup delays and provide evidence for the Sprint 1 getting-started and technology-stack criteria. The technology choices must also be motivated rather than merely listed.

Scope

Included

- testing the setup process from a clean clone;
- checking prerequisite versions;
- validating all documented commands;
- documenting frontend, backend, contracts and documentation setup;
- documenting required environment variables;
- documenting common setup problems;
- listing and motivating the project technology stack.

Excluded

- changing the selected frontend or backend frameworks without a separate decision;
- configuring Supabase;
- configuring authentication;
- deploying to Azure or Cloudflare;
- implementing product features.

Acceptance Criteria

- The repository setup is tested from a clean clone
- Required Node.js, npm and Python versions are documented
- Dependency installation instructions are verified
- Backend environment variables are documented
- Frontend environment variables are documented
- Commands for running the frontend are verified
- Commands for running the backend are verified
- Commands for testing, linting, formatting, type-checking and building are verified
- `apps/frontend/README.md` is created or completed
- `apps/backend/README.md` is created or completed
- The `packages/contracts` setup instructions are verified
- MkDocs local setup instructions are verified
- Common setup problems and solutions are documented
- A technology-stack document lists every major

framework, library, language and hosting service currently selected

- Each major technology includes its purpose and motivation
- Important alternatives considered are recorded where appropriate
- The new documentation is linked from the root README
- The new documentation is included in the MkDocs navigation
- A second team member follows the guide and records whether it works
- No unrelated product implementation is added

Dependencies

The issue may begin immediately.

It may only be closed once the CI and reproducible-installation issue confirms the final dependency installation and checking commands.

Evidence

Evidence must include:

- the clean-clone verification record;
- updated onboarding documentation;
- frontend and backend READMEs;
- the technology-stack document;
- second-team-member verification;
- updated MkDocs navigation;
- the linked Pull Request.

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

i want to do this issue now, need to follow project and git methodology please

in class he said we need to have every single thing we used documented or we will lose marks

what do u need from me to do this?

also, we have gotten a pro supabase account and are sticking to it

8/13/2026, 3:18:02 PM



Thinking



Remembering

- Reviewed COMS3011A methodologies, Git conventions, workflow, documentation, and AI tracking

✓ Worked for 14s

Yes. This is worth doing carefully because **Issue #12 directly targets three Sprint 1 rubric areas**: Getting Started / Dev Guides, Tech Stack, and indirectly

Documentation Site. For the top band, the brief says **all repos should have**

README/getting-started material and the **technology stack must be fully**

listed and motivated. `project_briefs.pdf` The project also specifically requires a non-monolithic frontend/backend, a hand-written API, established authentication, an external API integration, and a public documentation website. `project_briefs.pdf`

And yes: now that you have **Supabase Pro and are definitely keeping Supabase**, we should document **Supabase as a selected technology**, including exactly what you use it for and what you *do not* use it for. That distinction matters because the brief explicitly says you may not rely on Supabase-generated API endpoints in place of your own hand-written API. `project_briefs.pdf`

Before I write/change anything, I want to audit the **actual repository as it exists now**, rather than inventing a stack based on what we remember.

What I need from you

Please give me **one fresh export/ZIP of the repository from Gitea**, ideally from `main` **after your most recent merged PRs**.

That should let me inspect:

- root `package.json`
- every workspace `package.json`
- `package-lock.json`
- frontend source/config
- backend source/config
- `packages/contracts`
- `.gitea/workflows`
- ESLint configuration
- Prettier configuration
- TypeScript configuration
- Vitest/test configuration
- MkDocs config
- current READMEs
- environment example files
- Docker/config files if present
- Supabase-related dependencies/config
- Cloudflare/Wrangler files
- Azure files
- anything else already added to the repo

This is important because your lecturer's comment about **“every single thing we used”** means I don't want to give you a generic stack page saying “React, Express, Supabase”. We should derive the inventory from the actual repo and document even the supporting tooling where appropriate.

I also need these answers from you

Send me these in plain English; short answers are fine:

1. **Who will be the second team member who performs the clean-clone verification?**

Don't have them do it yet. We'll first produce the guide, then give them an exact test script.

2. **What OS are you developing on?**

For example Windows 11. If team members use multiple OSes, tell me.

3. **What IDE/editor(s) has the team actually used?**

VS Code? Qoder? IntelliJ? Something else?

This matters because if the lecturer literally wants all tooling documented, we should distinguish:

- required project technologies;
- development tooling;
- optional developer tooling.

4. **What AI tools have actually been used on this repository so far?**

I already know ChatGPT Web has been involved heavily, but tell me whether anybody has used:

- Qoder
- GitHub Copilot
- Claude Code
- Claude Web
- Codex
- Cursor
- any other AI autocomplete/code review tool.

The AI policy requires the tool, model and purpose to be tracked, and specifically says AI code-generation, inline editing and code-review usage/non-usage must be declared in the repository README. [ai_policy_20_jul.pdf](#)

5. **For Supabase Pro, what are you now using/planning to use it for?**

I believe:

- PostgreSQL/database
- Supabase Auth
- possibly storage?

Tell me whether Storage, Realtime, Edge Functions, etc. are being used. We should not claim services you aren't actually using.

6. **Are Azure and Cloudflare still the deployment plan?**

From our earlier work I understand roughly:

- frontend → Cloudflare
- backend → Azure
- database/auth → Supabase

Just tell me whether that remains correct.

7. **Have you actually used any external libraries beyond what is in**

`package.json` **manually**, e.g. copied snippets, templates, UI components, starter templates, Stack Overflow code, etc.?

This is particularly worth tracking now because Sprint 2 explicitly awards marks for **“All third-party code documented and motivated.”** `project_briefs.pdf`

We may as well build the habit now instead of scrambling later.

What I will produce from that

Once you send the repository ZIP + those answers, I'll go through it and give you the work **in your normal Git-methodology order**, not just dump documentation at you:

1. verify Issue #12 is correctly scoped and identify anything that should be adjusted now that Supabase Pro is confirmed;
2. tell you the correct branch name according to your methodology;
3. audit every technology/tool/dependency currently used;
4. separate **major stack, supporting libraries, development tooling, testing/quality tooling, hosting/infrastructure**, and **AI tooling**;

5. identify missing/onboarding docs;
6. walk you through a clean-clone test yourself;
7. create/update:
 - root README
 - apps/frontend/README.md
 - apps/backend/README.md
 - contracts setup docs
 - developer onboarding guide
 - technology-stack document
 - common troubleshooting section
 - environment-variable documentation
 - MkDocs navigation;
8. make sure **each technology has a purpose + motivation + alternatives where appropriate**;
9. produce the clean-clone evidence record;
- L0. give you the exact instructions for a **second team member** to independently verify it;
- L1. update the AI usage CSV/declaration and tell you which transcript evidence to save;
- L2. give you the commits in sensible atomic chunks;
- L3. give you the final PR text referencing #12 ;
- L4. only then tell you which acceptance criteria can be ticked and whether Issue #12 can actually close.

One important point: the issue currently says it **“may only be closed once the CI and reproducible-installation issue confirms the final dependency installation and checking commands.”** So we can complete essentially all of #12 now, but we should respect that dependency and not prematurely close it if that CI/reproducibility issue is still outstanding.

Also, for this chat itself: because we're using ChatGPT to plan/review/generate project documentation, it needs to form part of your AI evidence. The course policy explicitly allows AI for documentation and planning, but requires attribution, and says you remain responsible for verifying the output. [ai_policy_20_jul.pdf](#)

Send me the latest repo ZIP first. Once I have that, I'll audit what you've *actually* used before we touch the branch.

8/13/2026, 3:18:03 PM



Sport-Analytics-Tool (2)(1).zip

1. can we do a pr and then in that ask someone to clone test kind of thing once we done?
2. 2/ can we maybe do it, and then ask everyone to put what os they used (again, wont close the issue till everyone has done it and said they done it? or is this a mission? otherwise ill ask at the standup later
3. 3\ same as 2
4. should be in the ai registers and transcripts and readmes, u shouldl get that from there
5. read the docs
6. yes
7. idk

8/13/2026, 3:27:12 PM



Thinking



Inspecting ZIP archive contents

- Inspected archive contents and read methodology and deployment documentation

✓ Worked for 34s

1. **Yes - that is actually the cleanest way to handle the second-person verification.** We can do all the documentation on the `#12` branch, open the PR, and then ask another team member to do a genuine clean clone **from that PR branch** and follow the guide exactly. They then report what worked/failed, we fix anything on the same branch, they re-test, and only after verification do we merge and close `#12`. That gives you much stronger evidence than asking someone to test after it is already merged.

2. For **OS**, we can also make this part of the verification rather than you chasing everyone now. I would **not require all six people to do a full clean-clone test** - that's unnecessary work. One independent verifier satisfies the issue. But it would be useful to ask everyone at standup something simple like: "What OS and editor/IDE are you using for this project?" Then we can document the environments the team actually uses. We don't need to hold #12 open until every person has independently completed onboarding unless you deliberately want that extra evidence.
3. Same for the **IDE/editor**. Ask at standup. We can initially document the tooling visible in the repository and make an explicit distinction between:
 - required software;
 - recommended development tools;
 - team-used tools.

Nobody should be forced to install VS Code/Qoder/etc. merely because another team member uses it.

4. **You're right about the AI tools. I checked the repository rather than making you tell me again.** Your AI registers/transcripts and README already provide substantial evidence. For example, the root README currently records ChatGPT Web, Codex and Claude.ai usage, while the individual registers record task-level tool/model/purpose information. That aligns well with the course AI policy requirement to record tool, model and purpose. `ai_policy_20_jul.pdf`
5. **I also checked the Supabase/database docs and repository.** Supabase is definitely embedded in the current architecture, not merely proposed. The backend uses PostgreSQL directly through `pg`, uses `node-pg-migrate` for migrations, and includes `@supabase/supabase-js`; the frontend also uses `@supabase/supabase-js`. Your environment docs explicitly say the frontend Supabase values are for managed browser authentication, while application data remains behind your handwritten backend API. That separation is important because the project brief expressly prohibits using generated Supabase endpoints as your application API. `project_briefs.pdf`

So the move to **Supabase Pro** should become part of #12's technology motivation. We can document something along the lines of: Supabase provides the hosted PostgreSQL database and managed authentication; the paid project resolves the storage/capacity limitation encountered on the previous tier; application-domain access still goes through the team's handwritten Express API.

6. I found one thing we need to resolve while doing #12: **the repository currently says slightly different things about deployment.** The root README currently says both frontend and backend are deployed to **Azure App Service**, while the documentation site is on **Cloudflare Pages**. There are also separate docs for Azure frontend/backend and Cloudflare Pages.

So when you answered “yes” to my earlier `frontend → Cloudflare / backend → Azure` description, that does **not currently match the repo**.

From the actual current repository, it looks like:

```
Frontend application → Azure App Service
Backend API          → Azure App Service
Database/Auth        → Supabase
Documentation site    → Cloudflare Pages
CI/CD                → Gitea Actions
```

If that's the actual current decision, we'll document that. **Don't change anything now - just confirm that this is correct.**

7. For copied third-party code: **“idk” is fine.** We can audit what can be audited. Your `package.json` files give us a very good inventory of third-party packages, and we'll document/motivate those. We don't need to invent a claim that nobody has ever copied a snippet. At standup you could simply ask: “Has anyone copied or substantially adapted third-party code/templates/snippets into the repo outside normal npm packages?” If nobody knows of any, record that rather than claiming absolute certainty.

I've now looked through the ZIP

You already have **a lot more of #12 partially done than the issue makes it sound**. The repository currently includes:

- root `README.md` ;
- `docs/development/setup.md` ;
- `docs/development/dependencies.md` ;
- `docs/development/testing.md` ;
- `docs/environment.md` ;
- `database/README.md` ;
- `packages/contracts/README.md` ;
- MkDocs configuration;
- AI registers and transcripts;
- deployment documentation;
- `.env.example` files;
- ESLint;
- Prettier;
- TypeScript;
- Vitest;
- Playwright;
- axe accessibility testing;
- Redocly/OpenAPI tooling;
- Wrangler;
- React/Vite;
- Express;
- PostgreSQL tooling;
- Supabase;
- Zod contracts;
- security/logging middleware;
- and quite a bit more.

The main job for #12 is therefore **not “write onboarding from nothing.”** It is:

audit → consolidate → fill gaps → motivate every technology → independently verify

That is much better.

And it directly targets the Sprint 1 rubric: the Advanced band specifically requires **all repos to feature README/getting started** and a **fully listed, motivated technology stack**. `project_briefs.pdf`

What I think we should do next

According to your Git methodology, #12 already exists and is substantial documentation work, so we should now create a short-lived docs branch from updated `main`. Your methodology requires the chain:

```
Issue → branch → commits → PR → review → merge
```

and says substantial documentation gets its own branch.

For this issue I would use:

```
git switch main
git pull origin main
git switch -c docs/12-validate-developer-onboarding
```

Don't start editing yet.

First, I want to do a proper inventory of this ZIP and map:

```
technology/tool
→ exact version
→ where it appears
→ purpose
→ why selected
→ alternatives considered
→ whether developer must install it
→ whether docs already cover it
→ what #12 still needs
```

That will also answer your lecturer's "document every single thing" concern without us missing some boring-but-markable tool such as `cross-env`, `Supertest`, `jsdom`,

Redocly, axe, `node-pg-migrate`, `pino-http`, etc.

One thing only from you before I give you **Step 1**:

Confirm this current deployment architecture is correct: Azure frontend + Azure backend + Supabase database/auth + Cloudflare Pages documentation site.

Then we'll do #12 sequentially, including the branch, audit, exact files to edit, checks, AI register/transcript entry, commits, PR, and finally the clean-clone request to a teammate.

8/13/2026, 3:27:12 PM



yes, i think u correct, where does wrangler come in tho?

8/13/2026, 3:30:03 PM



Thinking



Inspecting Project Deployment Files

- Inspected deployment files and searched project for Cloudflare references

✓ Worked for 18s

Wrangler is for the documentation site, not the frontend app.

I checked the repo: the root README says the **MkDocs documentation is deployed to Cloudflare Pages using Wrangler**, and the actual deployment command is:

```
npx wrangler pages deploy site --project-name=sports-analytics-tool
```

The same command is documented in `docs/deployment/cloudflare_pages.md`. So your architecture is:

```
Frontend application → Azure App Service
Backend API          → Azure App Service
Database + Auth      → Supabase Pro
Documentation website → Cloudflare Pages
                        ↑
                    Wrangler CLI
```

Wrangler is basically **Cloudflare's command-line deployment tool**. In your project it builds/handles the bridge between the locally generated MkDocs `site/` folder and the public Cloudflare Pages project. It is listed as a root dev dependency at `wrangler ^4.119.0`.

So for #12, we should document it separately as something like:

Wrangler

- Purpose: CLI used to deploy the generated MkDocs documentation site to Cloudflare Pages.
- Why selected: provides a repeatable command-line deployment process compatible with the project's private Gitea repository.
- Not used for: deploying the React frontend or Express backend.

That distinction is actually a good example of why your lecturer wants the **whole stack documented**: "Cloudflare Pages" is the hosting service, while "Wrangler" is the tool you use to interact with/deploy to it.

Also, `pg-cloudflare` appearing in `package-lock.json` **does not mean you're using Cloudflare for PostgreSQL**. It is a transitive dependency brought in by the `pg` ecosystem, so we shouldn't misleadingly list it as one of *your selected technologies*. We should distinguish direct dependencies from transitive dependencies in the tech-stack audit.

So yes, your deployment picture is now confirmed. **Next we can actually start #12 with the branch and then do the technology inventory.**

8/13/2026, 3:30:04 PM



do it

8/13/2026, 3:30:45 PM



Thinking



Inspecting Project Repository Structure

- **Clarifying verification work**

I'm identifying documentation and tooling inconsistencies. I'll update READMEs, stack and setup guidance, record clean-clone verification, and distinguish local checks from checks requiring database or browser-test dependencies.

- **Clarified verification work**



Checking documentation and tooling consistency

- **Structuring documentation**

I'm defining the documentation set now: frontend, backend, contracts, setup, technology stack, and validation records. I'll separate completed checks from pending independent verification so the draft remains honest.

- **Structured documentation**

- **Audited repository structure, documentation, dependencies, configuration, workflows, and setup integrity**



Documenting project tooling and version requirements

- **Documented project tooling and audited AI-assisted code review usage**



Checking Repository Formatting Configuration

- **Reviewed repository configuration, documentation, and onboarding setup needs**



Updating root README AI usage categories

- **Updated root README AI usage categories**

- **Patched README documentation for AI usage**
 - **Auditing AI declarations and deployment documentation**
 - **Audited AI declarations and deployment documentation**
 - **Updated Azure documentation, AI review wording, and validated project dependencies**
 - **Validating documentation links and YAML paths**
 - **Validated MkDocs YAML configuration and documentation navigation paths**
 - **Searching documentation for CI reproducibility references**
 - **Updated and validated CI, onboarding, deployment, and repository documentation patches**
- ✓ Worked for 21m 34s

Done. I audited the latest repository ZIP and prepared the **Issue #12 documentation changes against the current** `main` (`ee74b86`).

The patch adds/completes the frontend README, backend README, contracts README, full clean-clone onboarding guide, environment-variable documentation, troubleshooting, an exhaustive technology-stack inventory, deployment/tooling documentation, MkDocs navigation, root README links, and an onboarding verification record. I also made the structure check require the new onboarding documents.

That directly addresses the Sprint 1 criteria for **all repos having README/getting-started documentation** and a **fully listed and motivated technology stack**.

`project_briefs.pdf` I also corrected the repository-level AI declaration so it agrees with the AI registers; the course policy specifically requires repository declarations for AI code generation, inline editing and code review. `ai_policy_20_jul.pdf`

1. Start the branch

First make sure Issue #12 is assigned, in Sprint 1, and move it to **In Progress**. Then:

```
git switch main
git pull origin main
```

```
git switch -c docs/12-validate-developer-onboarding
```

2. Apply the prepared changes

Download this file:

[Download the Issue #12 onboarding.patch](#)

Put it in your repository root and run:

```
git apply --check issue-12-onboarding.patch
git apply issue-12-onboarding.patch
```

Then delete the patch file itself so you don't accidentally commit it:

```
rm issue-12-onboarding.patch
```

If you're using PowerShell:

```
Remove-Item issue-12-onboarding.patch
```

Then:

```
git status
```

You should see **18 changed/new repository files**.

I separately verified that the patch applies cleanly to the exact `main` from your ZIP, that `git diff --check` passes, and that:

```
node scripts/check-required-files.mjs
```

now reports:

```
Repository structure check passed (24 required files).
```

I have **not** falsely filled in the clean-clone evidence or claimed `npm run check` passed on your machine. Those need actual human execution.

3. What is now documented

The new technology-stack page is deliberately exhaustive. It covers the obvious technologies **and the smaller direct dependencies**, including React, React Router, Vite, Express, TypeScript, Node, PostgreSQL, Supabase, Zod, `pg`, CORS, Helmet, Pino HTTP, dotenv, `node-pg-migrate`, Vitest, Testing Library, jsdom, Supertest, Playwright, axe, ESLint, Prettier, `cross-env`, `tsx`, Redocly, MkDocs Material, Wrangler, Git, Gitea, Gitea Actions, Azure App Service, Cloudflare Pages, Cricsheet and Google OAuth.

It also explains an important distinction:

```
Frontend application → Azure App Service
Backend API         → Azure App Service
Database            → Supabase-hosted PostgreSQL
Authentication      → Supabase Auth
Documentation site   → Cloudflare Pages
Docs deployment tool → Wrangler
```

And it explicitly records that **Wrangler is for MkDocs → Cloudflare Pages**, not your React frontend.

I also found and corrected stale documentation saying Azure had not yet been selected even though ADR 0003 already approves Azure App Service.

4. Don't commit everything as one giant commit

Follow your Git methodology. I recommend these four commits first.

Commit 1 - onboarding guides

```
git add apps/frontend/README.md
git add apps/backend/README.md
git add packages/contracts/README.md
git add docs/development/setup.md
git add docs/development/dependencies.md
git add docs/environment.md

git commit -m "docs(onboarding): document reproducible developer setup" -m
"Refs #12"
```

Commit 2 - technology stack and deployment alignment

```
git add docs/development/technology-stack.md
git add README.md
git add docs/index.md
git add mkdocs.yml
git add docs/deployment/overview.md
git add docs/deployment/azure.md
git add docs/deployment/azure-backend.md
git add docs/deployment/azure-fronted.md
git add infra/azure/README.md

git commit -m "docs(stack): document and motivate selected technologies" -m
"Refs #12"
```

Commit 3 - enforce the documentation structure

This one changes the `.mjs` validation script, so because AI generated that code change, include the required code attribution:

```
git add scripts/check-required-files.mjs

git commit -m "chore(structure): require onboarding documentation" -m "Refs
#12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

Commit 4 - verification evidence template

```
git add evidence/validation/issue-12-onboarding-verification.md

git commit -m "docs(evidence): add onboarding verification record" -m "Refs
#12"
```

Leave `docs/ai/usage.md` uncommitted for the moment. We'll bundle that with **your Issue #12 AI register entry + this chat transcript** as the final AI-evidence commit.

5. Then verify before we open the draft PR

After those commits, run:

```
npm run format:check
npm run check
```

```
python -m mkdocs build --strict
git diff --check
git status
```

Also start both applications once:

```
npm run dev:backend
```

and in another terminal:

```
npm run dev:frontend
```

Check:

```
http://localhost:3000/api/v1/health
http://localhost:5173
```

Do not fill the independent-verifier section yet. That is deliberately left Pending.

Once you've done these commands, send me the outputs - especially if anything fails. Then I'll give you the **AI CSV entry + transcript filename + final AI commit**, after which we push and open a **Draft PR #12 specifically so another team member can clean-clone that branch and verify the guide.**

That approach gives you actual methodology evidence instead of merely claiming the onboarding works.

8/13/2026, 3:30:47 PM



i need to prettier alot of files, give me command (my machine needs npm.cmd or mpx.cmd etc)

Checking formatting...

[warn] apps/backend/README.md

[warn] apps/frontend/README.md

[warn] docs/deployment/azure-backend.md

[warn] docs/deployment/azure-fronted.md

[warn] docs/deployment/azure.md

[warn] docs/deployment/overview.md

[warn] docs/development/setup.md

[warn] docs/development/technology-stack.md

[warn] docs/environment.md

[warn] evidence/decisions/ADR-003-database-host-connection-and-migrations.md

[warn] evidence/decisions/ADR-005-database-host-migration.md

[warn] Code style issues found in 11 files. Run Prettier with --write to fix.

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>

i havent done any commits yet

8/13/2026, 3:58:02 PM



Since you **haven't committed yet**, I'd format only the files Prettier flagged so we don't create unrelated changes across the repo.

Run this in PowerShell:

```
npx.cmd prettier --write apps/backend/README.md apps/frontend/README.md
docs/deployment/azure-backend.md docs/deployment/azure-fronted.md
docs/deployment/azure.md docs/deployment/overview.md docs/development/setup.md
docs/development/technology-stack.md docs/environment.md
evidence/decisions/ADR-003-database-host-connection-and-migrations.md
evidence/decisions/ADR-005-database-host-migration.md
```

Then check again:

```
npm.cmd run format:check
```

If that passes, run:

```
git status
```

One thing: those two ADR files were not part of the #12 changes I listed earlier, so after formatting I want to check whether Prettier changed them only cosmetically or whether they were already unformatted on `main`. **Don't commit anything yet**—send me the `git status` output after formatting and I'll tell you exactly what belongs in the #12 commits.

8/13/2026, 3:58:10 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> python -m mkdocs build --strict
```

```
|  ⚠ Warning from the Material for MkDocs team
|
|  MkDocs 2.0, the underlying framework of Material
for MkDocs,
|  will introduce backward-incompatible changes,
including:
|
|  × All plugins will stop working - the plugin system
has been removed
|  × All theme overrides will break - the theming
system has been rewritten
|  × No migration path exists - existing projects
cannot be upgraded
|  × Closed contribution model - community members
can't report bugs
|  × Currently unlicensed - unsuitable for production
use
|
|  Our full analysis:
|
|  https://squidfunk.github.io/mkdocs-
material/blog/2026/02/18/mkdocs-2.0/
```

```
INFO - Cleaning site directory
```

```
INFO - Building documentation to directory:
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\site
```

```
INFO - The following pages exist in the docs
directory, but are not included in the "nav"
configuration:
```

- adr\0003-azure-hosting.md
- data\cricsheet.md
- requirements\sport-domain-definition.md

```
INFO - Documentation built in 3.14 seconds
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

also, did u change anything else that i should test b4 committing

8/13/2026, 3:59:49 PM



Thinking



Reviewing Patch Changes and Test Coverage

- **Assessing documentation tests**

I'm reading the successful build as confirmation that the new pages are valid and already reachable through navigation. The three informational messages indicate optional existing pages, not failures. I'm checking whether the broader validation command is still worthwhile.

- **Assessed documentation tests**

- **Reviewed patch changes, npm scripts, and test coverage**

✓ Worked for 48s

Your MkDocs result is **good** — `build --strict` completed successfully. The Material message is an upstream warning about future MkDocs 2.0, not a problem with your repo.

The three pages not in `nav` are also only informational:

```
adr/0003-azure-hosting.md
data/cricsheet.md
requirements/sport-domain-definition.md
```

They do **not** make the build fail. We can decide later whether to add them to the navigation; they weren't introduced by #12.

As for what I changed: **I did not change application functionality.** No React/Express feature code, database migration, `package.json`, CI workflow, or environment-example values were changed.

The only actual executable code I changed was:

```
scripts/check-required-files.mjs
```

I added the new onboarding docs to its required-file list, so the repository structure check now verifies that these exist:

```
apps/frontend/README.md
apps/backend/README.md
packages/contracts/README.md
docs/development/technology-stack.md
```

Everything else is documentation/configuration/navigation.

Before committing, do these checks

Since you're on Windows and need `.cmd`, first make sure Prettier now passes:

```
npm.cmd run format:check
```

Then run the **full repository gate**:

```
npm.cmd run check
```

That is the important one. It runs:

```
structure check
→ Prettier
→ ESLint
→ contracts build
→ TypeScript typecheck
→ unit/frontend/API/contracts tests
→ OpenAPI lint
→ full build
```

So if:

```
npm.cmd run check
```

passes, we've tested the only executable change I made as well as ensuring the documentation edits didn't break the repository gate.

You've already passed:

```
python -m mkdocs build --strict
```

So MkDocs is covered.

Then I would run:

```
git diff --check
```

and:

```
git status
```

One more useful test

Because #12 specifically documents the developer commands, start each app once **if your `.env` files are already configured:**

Terminal 1:

```
npm.cmd run dev:backend
```

Check:

```
http://localhost:3000/api/v1/health
```

Terminal 2:

```
npm.cmd run dev:frontend
```

Check:

```
http://localhost:5173
```

We're **not** doing the true clean-clone test yet. That's exactly what the teammate should do from the PR branch later, which gives us independent evidence for #12.

So your immediate sequence is:

```
npm.cmd run format:check  
npm.cmd run check  
git diff --check  
git status
```

You've already done the MkDocs strict build successfully.

Don't commit yet if `npm.cmd run check` **fails.** Send me the output and we'll fix it first.

8/13/2026, 3:59:51 PM



vite v5.4.21 building for production...

✓ 136 modules transformed.

dist/index.html 0.63 kB | gzip: 0.36 kB

dist/assets/index-D90AxV8D.css 16.53 kB | gzip: 3.89 kB

dist/assets/index-U9MTeCjg.js 501.53 kB | gzip: 139.48 kB

(!) Some chunks are larger than 500 kB after minification. Consider:

- Using dynamic import() to code-split the application
- Use build.rollupOptions.output.manualChunks to improve chunking: <https://rollupjs.org/configuration-options/#output-manualchunks>
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.

✓ built in 2.12s

is this something we caused?

8/13/2026, 4:02:58 PM



Thinking



Inspecting Patch Changes for Bundle Warning

- Inspected patch changes for bundle warning

✓ Worked for 11s

No — **that warning was not caused by the #12 changes.**

Our patch did not touch any frontend source code, Vite config, dependencies, or build settings. It only changed documentation plus the required-files structure check. I rechecked the patch: nothing under `apps/frontend/src`, no `vite.config.*`, and no `package.json` changes.

This part is the key:



```
dist/assets/index-U9MTeCjg.js    501.53 kB
```

Vite warns because the generated JS bundle is **just over its 500 kB warning threshold**. The build still succeeded:

```
✓ built in 2.12s
```

So for #12, I would treat this as:

- **not a failure**
- **not introduced by this branch**
- **not something we need to fix before committing #12**

It could be worth creating a separate performance/technical-debt issue later for frontend code-splitting if you want, but I would **not mix that into Issue #12**.

If `npm.cmd run check` finished successfully after this warning, then that check has passed.

8/13/2026, 4:03:01 PM



(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

- modified: README.md
- modified: docs/ai/usage.md
- modified: docs/deployment/azure-backend.md
- modified: docs/deployment/azure-fronted.md
- modified: docs/deployment/azure.md
- modified: docs/deployment/overview.md
- modified: docs/development/dependencies.md
- modified: docs/development/setup.md
- modified: docs/environment.md
- modified: docs/index.md
- modified: evidence/decisions/ADR-003-database-host-connection-and-migrations.md
- modified: evidence/decisions/ADR-005-database-host-migration.md
- modified: infra/azure/README.md
- modified: mkdocs.yml
- modified: packages/contracts/README.md
- modified: scripts/check-required-files.mjs
- modified: site/404.html
- modified: site/ai/usage/index.html
- modified: site/api/overview/index.html
- modified: site/architecture/overview/index.html
- modified: site/architecture/repository-structure/index.html
- modified: site/database/overview/index.html
- modified: site/database/schema/index.html
- modified: site/deployment/azure/index.html
- modified: site/deployment/overview/index.html
- modified: site/development/dependencies/index.html
- modified: site/development/setup/index.html
- modified: site/development/testing/index.html
- modified: site/git-methodology/index.html
- modified: site/index.html
- modified: site/project_methodology/index.html

```
modified: site/search/search_index.json
modified: site/security/overview/index.html
modified: site/sitemap.xml.gz
```

Untracked files:

(use "git add <file>..." to include in what will be committed)

```
apps/backend/README.md
apps/frontend/README.md
docs/development/technology-stack.md
evidence/validation/issue-12-onboarding-
verification.md
site/adr/
site/ai_policy.pdf
site/api/contracts/
site/api/openapi.yaml
site/api/openapi/
site/api/public-read/
site/api/versioning/
site/architecture/system-architecture/
site/data/
site/database/access/
site/database/erd/
site/deployment/azure-backend/
site/deployment/azure-fronted/
site/deployment/cloudflare_pages/
site/design/
site/development/technology-stack/
site/environment/
site/planning/
site/requirements/
site/security/auth-provider-comparison/
site/security/authentication/
```

no changes added to commit (use "git add" and/or "git commit -a")

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

help commit these

8/13/2026, 4:06:09 PM



Thinking



Inspecting Repository Gitignore and Generated Site Files

- **Assessing site changes**

I'm deciding whether generated site files belong in this commit. Since deployment can rebuild from source and the existing artifact is stale, I'm leaning toward restoring tracked files and removing untracked output to keep the change focused.

- **Assessed site changes**



Inspecting Issue #12 Patch Files

- **Planning cleanup**

I'm separating generated-site cleanup from formatting fixes. Restoring `site/` keeps artifacts out of the feature commit, while the two reformatted decision records may need their own focused commit if the repository's formatting check requires them.

- **Planned cleanup**



Reviewing Documentation and Deployment Diffs

- **Inspected repository history, applied onboarding patch, and reviewed documentation deployment diffs**

✓ Worked for 2m 53s

Yes. **Do not** `git add .` here because the MkDocs build generated all those `site/` changes.

First clean the generated `site/` output. It was created/refreshed by `mkdocs build --strict`; we don't need it in this #12 PR:

```
git restore site
git clean -fd -- site
```

Then:

```
git status
```

The `site/...` entries should be gone.

Keep the two ADR changes. Those are the files Prettier found already failing formatting. Since #12 is validating that `npm.cmd run check` actually works, fixing those two formatting-only problems is reasonable - we'll isolate them in their own commit so it is transparent.

Commit 1 - onboarding documentation

Stage:

```
git add apps/backend/README.md
git add apps/frontend/README.md
git add packages/contracts/README.md
git add docs/development/dependencies.md
git add docs/development/setup.md
git add docs/environment.md
```

For `README.md`, we want the onboarding changes now but leave the AI-declaration change for its own commit:

```
git add -p README.md
```

You should get **3 hunks**. Answer:

```
1st hunk: y
2nd hunk: y
3rd hunk (AI usage): n
```

Check what is staged:

```
git diff --cached --name-only
```

Then:

```
git commit -m "docs(onboarding): document reproducible developer setup" -m
"Refs #12"
```

Commit 2 - full technology stack + deployment alignment

```
git add docs/development/technology-stack.md
git add docs/deployment/azure-backend.md
git add docs/deployment/azure-fronted.md
git add docs/deployment/azure.md
git add docs/deployment/overview.md
git add docs/index.md
git add infra/azure/README.md
git add mkdocs.yml

git diff --cached --name-only
```

Then:

```
git commit -m "docs(stack): document and align selected technologies" -m "Refs #12"
```

This is the important lecturer-facing commit: it documents/motivates the actual stack and aligns Azure, Supabase, Cloudflare Pages and Wrangler.

Commit 3 - repository structure validation

This is the **one actual code/config script change** I generated, so it needs the AI trailer:

```
git add scripts/check-required-files.mjs

git diff --cached
```

Then:

```
git commit -m "chore(structure): require onboarding documentation" -m "Refs #12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

Commit 4 - the two existing Prettier failures

Don't hide these inside another commit. Record exactly what happened:

```
git add evidence/decisions/ADR-003-database-host-connection-and-migrations.md
git add evidence/decisions/ADR-005-database-host-migration.md
```

```
git diff --cached
```

That diff should be **formatting only**. Then:

```
git commit -m "chore(format): format existing decision records" -m "Refs #12"
```

This is useful evidence: clean-clone/check validation exposed pre-existing formatting failures, and you corrected them.

Commit 5 - verification record

```
git add evidence/validation/issue-12-onboarding-verification.md

git commit -m "docs(evidence): add onboarding verification record" -m "Refs #12"
```

We're deliberately committing it with the independent-verifier fields still **Pending**. We fill those from real results once someone tests the PR.

Commit 6 - AI documentation reconciliation

At this point the only **README.md** change left should be its AI section.

```
git add README.md
git add docs/ai/usage.md

git diff --cached
```

Then:

```
git commit -m "docs(ai): reconcile repository AI usage declarations" -m "Refs #12"
```

The course's AI policy requires the repository declaration to distinguish code generation, inline editing and code review, so this is worth keeping as an explicit commit rather than burying it. **ai_policy_20_jul.pdf**

Then check where you are

```
git status
git log --oneline -6
```

You should have roughly:

```
docs(ai): reconcile repository AI usage declarations
docs(evidence): add onboarding verification record
chore(format): format existing decision records
chore(structure): require onboarding documentation
docs(stack): document and align selected technologies
docs(onboarding): document reproducible developer setup
```

Don't push/open the PR just yet. We still need to add **your #12 AI-register entry and this chat transcript**, and then we can push and open the draft PR for someone else to clean-clone and test.

If `git add -p README.md` looks confusing when you get there, paste what PowerShell shows and I'll tell you exactly which key to press.

8/13/2026, 4:06:10 PM



give commit messages again with Ai declaration in them, ie Assisted by

8/13/2026, 4:10:59 PM



Yep - use the AI declaration on **each commit** for this issue since ChatGPT assisted with the documentation/planning.

Use:

```
Assisted-by: ChatGPT-Web[GPT-5.6 Sol]
```

Your commits:

```
git commit -m "docs(onboarding): document reproducible developer setup" -m  
"Refs #12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

```
git commit -m "docs(stack): document and align selected technologies" -m "Refs  
#12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

```
git commit -m "chore(structure): require onboarding documentation" -m "Refs  
#12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

```
git commit -m "chore(format): format existing decision records" -m "Refs #12" -  
m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

```
git commit -m "docs(evidence): add onboarding verification record" -m "Refs  
#12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

```
git commit -m "docs(ai): reconcile repository AI usage declarations" -m "Refs  
#12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

That gives each commit a clear purpose, links it to **#12**, and records the AI assistance consistently with your course policy. [ai_policy_20_jul.pdf](#)

After the six commits:

```
git log --format=full -6
```

That will let you verify the `Assisted-by:` trailer actually appears on every commit.

8/13/2026, 4:11:06 PM



when should i upload this transcript? and update the line in my ai register? is that commit 7?

8/13/2026, 4:14:44 PM



Thinking

- **Timing transcript export**

I'm recommending export after implementation and verification, but before opening the PR, so the transcript covers the substantive work. If later AI-assisted changes occur, I'd add a separate transcript and register entry.

- **Planned transcript export timing**

✓ Worked for 12s

Yes — **make that Commit 7**, and do it **after the six content commits but before you push/open the draft PR**.

That gives you a clean separation:

- 1 onboarding docs
- 2 technology stack
- 3 structure check
- 4 formatting fixes
- 5 verification template
- 6 repository AI declaration
- 7 AI usage evidence for Issue #12

For Commit 7, do two things together:

1. **Export/upload this ChatGPT conversation transcript** into your normal personal transcript folder.
2. **Add/update one line in your AI usage register** recording this Issue #12 work.

The register entry should describe the actual use broadly enough to cover what we did, e.g. planning the issue, auditing the repo, generating/reviewing onboarding and

technology-stack documentation, troubleshooting verification, and planning commits.

Then stage **only** the transcript and AI-register file and commit them together:

```
git add <your-ai-register-file>
git add <your-transcript-file>

git commit -m "docs(ai): record AI assistance for onboarding validation" -m
"Refs #12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

One important timing point

Don't export the transcript **right this second** if we're still going to use this same chat to finish the initial #12 work. Wait until we've finished:

```
commits
→ final checks
→ transcript/register
→ Commit 7
→ push
→ Draft PR
```

That way the transcript captures the whole initial implementation process.

Then, when the teammate clean-clone tests the PR, if you come back to ChatGPT and we fix problems based on their feedback, that is **new AI-assisted work**. We should then add either:

- another transcript + another register row; or
- an additional transcript covering that later session,

and make another small AI-evidence commit before the PR is finally merged.

So **yes: Commit 7 is the AI evidence commit for this current session.**

8/13/2026, 4:14:57 PM



give me the line to add to my csv and what to call this transcript
and give me the PR- make sure to state people need to add stuff if they do and only use closesd #12 if it actually does

8/13/2026, 4:15:52 PM



Thinking



Inspecting AI register and methodology files

- **Preparing PR text**

I'm drafting a concise pull-request description that documents scope, AI assistance, verification status, contributor follow-up, and the conditional relationship to issue #12. I'll keep the CSV update separate and avoid claiming completion before review.

- **Prepared PR text**

- **Documenting the change**

I'm preparing the requested CSV entry, commit command, and PR draft. I'm distinguishing completed checks from pending clean-clone verification, avoiding unsupported claims about application startup, and linking the evidence transcript clearly.

- **Documented the change**

- **Inspected AI register and methodology files**

✓ Worked for 48s

Yes - make this **Commit 7**, then push and open the PR as a **Draft** because #12 is not ready to close yet.

Use this transcript filename:

2026-08-13-issue-12-developer-onboarding-tech-stack.pdf

Save it here:

```
evidence/ai/transcripts/shayna-underslak/2026-08-13-issue-12-developer-onboarding-tech-stack.pdf
```

CSV line

Add this as the next row in:

```
evidence/ai/registers/shayna-underslak.csv
```

```
"2026-08-13","Shayna Underslak","ChatGPT Web","GPT-5.6 Sol","Planning; documentation generation; repository audit; technical guidance; Git and Gitea guidance; debugging","Issue #12: validate developer onboarding and motivate the technology stack","Developer onboarding and clean-clone documentation; frontend, backend and contracts README updates; environment and troubleshooting guidance; complete motivated technology-stack inventory; deployment documentation alignment; MkDocs navigation updates; onboarding verification record; repository structure-check update; AI declaration reconciliation; commit and Pull Request guidance","Reviewed the generated documentation against the current repository configuration, package manifests, existing architecture, deployment and AI documentation, and course requirements; ran repository quality checks and a strict MkDocs build; reviewed the Vite bundle-size warning as non-blocking and unrelated to the documentation changes; independent clean-clone verification remains pending before Issue #12 can be closed","evidence/ai/transcripts/shayna-underslak/2026-08-13-issue-12-developer-onboarding-tech-stack.pdf; Issue #12; linked Pull Request"
```

Then Commit 7:

```
git add evidence/ai/registers/shayna-underslak.csv
git add evidence/ai/transcripts/shayna-underslak/2026-08-13-issue-12-developer-onboarding-tech-stack.pdf

git commit -m "docs(ai): record AI assistance for onboarding validation" -m "Refs #12" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

Then push:

```
git push -u origin docs/12-validate-developer-onboarding
```

Open it as a **Draft Pull Request**.

```
:::writing{variant="document" id="58214" title="#12 Validate developer onboarding and motivate the technology stack"}
```

Purpose

Validate the project developer-onboarding process and improve the repository documentation so that a team member can set up, run, test and understand the project from a clean clone.

This Pull Request also consolidates and motivates the technologies currently used by the project, including the distinction between application hosting, database/authentication services, documentation hosting and development tooling.

The work supports the Sprint 1 Getting Started / Dev Guides, Technology Stack and Documentation Site criteria.

Changes

This Pull Request:

- adds and completes developer setup documentation for the frontend, backend and shared contracts package;
- expands the clean-clone setup instructions;
- documents required Node.js, npm and Python prerequisites;
- documents frontend and backend environment variables;
- documents dependency installation and workspace-specific commands;
- documents commands for development, testing, formatting, linting, type-checking and building;
- adds common setup problems and troubleshooting guidance;
- adds a technology-stack document covering the project's selected technologies and tooling;
- records the purpose and motivation for major technologies and relevant alternatives;
- documents the current deployment responsibilities:
 - React frontend → Azure App Service;

- Express backend → Azure App Service;
- PostgreSQL database → Supabase Pro;
- authentication → Supabase Auth;
- MkDocs documentation site → Cloudflare Pages;
- documentation deployment → Wrangler;
- clarifies that Supabase is not being used to replace the project's handwritten application API;
- aligns existing Azure and deployment documentation with the current architecture;
- links the onboarding and technology-stack documentation from the repository documentation;
- updates MkDocs navigation;
- updates the repository structure check to require the onboarding documentation;
- adds an Issue #12 onboarding-verification evidence record;
- formats two existing ADR files that were identified by the repository formatting check;
- reconciles repository-level AI usage documentation with the existing AI evidence registers; and
- adds the AI evidence associated with this work.

Testing and Verification

The following local verification has been performed by the author:

- ☒ `npm run format:check`
- ☒ `npm run check`
- ☒ `python -m mkdocs build --strict`
- ☒ Documentation site builds successfully
- ☒ Existing application production builds complete successfully
- ☒ Repository structure validation includes the newly required onboarding files
- ☒ Changed files were reviewed by the author

- ☒ No generated `site/` output is intentionally included in this Pull Request

The Vite production build currently reports a bundle-size warning for a JavaScript chunk slightly above 500 kB. The build succeeds, and this warning was not introduced by the Issue #12 documentation changes.

Independent clean-clone verification required

This Pull Request must remain a Draft and must not be merged yet.

At least one team member other than the author must perform a genuine clean-clone test of this branch and follow the onboarding documentation without relying on an existing configured checkout.

The verifier should record:

- operating system and version;
- editor/IDE used;
- Node.js version;
- npm version;
- Python version;
- whether dependency installation succeeded;
- whether frontend setup and startup succeeded;
- whether backend setup and startup succeeded;
- whether the contracts setup worked;
- whether the documented repository checks worked;
- whether MkDocs setup and local build worked;
- any unclear, missing or incorrect instructions; and
- any changes required as a result of the test.

The results must be added to:

`evidence/validation/issue-12-onboarding-verification.md`

If verification identifies a problem, it should be fixed on this branch and re-tested before approval.

Other team members are also encouraged to record the operating systems and editors/IDEs they actually use so the documentation reflects the team's real development environments.

Contributor evidence reminder

If another team member changes files in this Pull Request, they must ensure their contribution follows the repository Git methodology.

If AI is used to make, review or substantially assist with those changes, that team member must:

- update their own AI usage register;
- retain the relevant transcript/evidence where required;
- use the appropriate `Assisted-by:` commit trailer; and
- ensure the repository-level AI documentation remains accurate if a new AI tool or usage category is introduced.

Do not attribute AI assistance that was not actually used.

Known Limitations

Independent second-team-member clean-clone verification is still pending.

Issue #12 also states that it may only be closed once the dependent CI/reproducible-installation work has confirmed the final dependency-installation and repository-check commands.

For those reasons, this Draft Pull Request currently **references** Issue #12 rather than closing it.

Related Issue

Refs #12

Before final merge: once all Issue #12 acceptance criteria are genuinely complete, the independent verification has passed, required follow-up changes have been incorporated, and the dependency condition has been satisfied, change this line to:

`Closes #12`

Do not use `closes #12` while those requirements remain outstanding.

Review Checklist

- ☐ All applicable Issue #12 acceptance criteria are met
- ☒ The Pull Request contains no unrelated product implementation
- ☒ Security implications were considered where relevant
- ☒ Documentation and evidence were updated
- ☒ No credentials, secrets or sensitive information were intentionally committed
- ☒ AI assistance was attributed where required
- ☐ Independent clean-clone verification has been completed
- ☐ Any problems found during clean-clone verification have been resolved
- ☐ The dependent CI/reproducible-installation requirement has been satisfied
- ☐ At least one team member other than the author has reviewed and approved the Pull Request
- ☐ All reviewer comments have been resolved
- ☐ Refs #12 has been changed to Closes #12 only if the issue can actually close

Screenshots, API Examples or Other Evidence

No application UI or API behaviour is introduced by this Pull Request.

Relevant evidence includes:

- `evidence/validation/issue-12-onboarding-verification.md`
- `evidence/ai/registers/shayna-unterslak.csv`
- `evidence/ai/transcripts/shayna-unterslak/2026-08-13-issue-12-developer-onboarding-tech-stack.pdf`
- updated developer onboarding documentation
- updated technology-stack documentation
- local repository checks and MkDocs strict build

Additional independent-verification evidence will be added to this Pull Request before it is marked ready for final review.

AI Declaration

The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Sol].

:::

One small thing when you open it: **leave it as** `Refs #12` **now**. Once the teammate has done the clean-clone test and the dependency condition is genuinely satisfied, we can update the PR from `Refs #12` → `Closes #12` immediately before final approval/merge.

8/13/2026, 4:15:55 PM



give step by step of what teammate needs to follow inside the pr, so redo with that added

8/13/2026, 4:18:15 PM



Thinking

- **Drafting PR instructions**

I'm revising the PR description to include reproducible teammate steps: start from a clean clone, follow documented setup, run validation commands, record missing prerequisites, and avoid committing secrets or generated files.

- **Drafted PR instructions**

✓ Worked for 13s

Yes - I'd put the **exact clean-clone test procedure inside the PR**, so the teammate does not have to guess what "verify onboarding" means.

Use this revised PR description:

```
:::writing{variant="document" id="48317" title="#12 Validate developer onboarding and motivate the technology stack"}
```

Purpose

Validate the complete developer-onboarding process and improve the repository documentation so that a team member can set up, run, test and understand the project from a genuinely clean clone.

This Pull Request also consolidates and motivates the technologies currently used by the project, including the distinction between application hosting, database/authentication services, documentation hosting and development tooling.

The work supports the Sprint 1:

- Getting Started / Dev Guides;
- Technology Stack; and

- Documentation Site

criteria.

Changes

This Pull Request:

- adds and completes developer setup documentation for the frontend, backend and shared contracts package;
- expands the clean-clone setup instructions;
- documents required Node.js, npm and Python prerequisites;
- documents frontend and backend environment variables;
- documents dependency installation and workspace-specific commands;
- documents commands for development, testing, formatting, linting, type-checking and building;
- adds common setup problems and troubleshooting guidance;
- adds a technology-stack document covering the project's selected technologies and tooling;
- records the purpose and motivation for major technologies and relevant alternatives;
- documents the current deployment responsibilities:
 - React frontend → Azure App Service;
 - Express backend → Azure App Service;
 - PostgreSQL database → Supabase Pro;
 - authentication → Supabase Auth;
 - MkDocs documentation site → Cloudflare Pages;
 - documentation deployment → Wrangler;
- clarifies that Supabase is not being used to replace the project's handwritten application API;
- aligns existing Azure and deployment documentation with the current architecture;

- links the onboarding and technology-stack documentation from the repository documentation;
- updates MkDocs navigation;
- updates the repository structure check to require the onboarding documentation;
- adds an Issue #12 onboarding-verification evidence record;
- formats two existing ADR files identified by the repository formatting check;
- reconciles repository-level AI usage documentation with the existing AI evidence registers; and
- adds the AI evidence associated with this work.

Author Verification

The following local verification has been performed on this branch:

- ☒ `npm run format:check`
- ☒ `npm run check`
- ☒ `python -m mkdocs build --strict`
- ☒ Documentation site builds successfully
- ☒ Existing application production builds complete successfully
- ☒ Repository structure validation includes the newly required onboarding files
- ☒ Changed files were reviewed
- ☒ No generated `site/` output is intentionally included in this Pull Request

The Vite production build currently reports a bundle-size warning for a JavaScript chunk slightly above 500 kB. The build succeeds, and this warning was not introduced by the Issue #12 documentation changes.

Independent Clean-Clone Verification Required

This Pull Request must remain a Draft until the independent onboarding test has been completed.

A team member other than the author must follow the procedure below from a completely new clone.

The purpose is not simply to prove that the application works on another computer. The purpose is to verify that a developer who starts with the repository documentation can successfully configure and use the project.

Rules for the verifier

Please:

- use a completely new clone of the repository;
- do not reuse the author's existing project directory;
- do not copy an existing `node_modules` directory;
- do not copy existing local configuration except for the actual environment values required by the documentation;
- follow the repository documentation rather than relying on undocumented instructions from another team member;
- note anything that is confusing, missing, incorrect or outdated;
- do not commit `.env` files, passwords, keys or other secrets.

If something fails, **please record the failure rather than silently finding your own workaround**. A workaround that is missing from the documentation is useful evidence that the onboarding guide still needs improvement.

Clean-Clone Test Procedure

Step 1 - Record your development environment

Before starting, record:

- your name;
- operating system and version;
- editor or IDE used.

Check the installed tool versions:

```
node --version
npm --version
```

```
python --version
git --version
```

Record the versions you actually used.

If you are using Windows PowerShell and normal `npm / npx` commands do not execute correctly on your machine, use the Windows command variants where required, for example:

```
npm.cmd --version
npx.cmd --version
```

Do **not** install different versions simply to match another team member unless the onboarding documentation specifically requires them.

Step 2 - Create a genuinely clean clone

Clone the repository into a new directory.

```
git clone <repository-url>
cd Sport-Analytics-Tool
```

Fetch the Pull Request branch and switch to:

```
git fetch origin
git switch docs/12-validate-developer-onboarding
```

Confirm that you are testing the correct branch:

```
git branch --show-current
```

Expected:

```
docs/12-validate-developer-onboarding
```

Also confirm that the working tree begins clean:

```
git status
```

Step 3 - Read the onboarding documentation first

Before asking another team member for setup instructions, follow the repository documentation.

Start with:

```
README.md
docs/development/setup.md
docs/development/dependencies.md
docs/environment.md
```

Also review the relevant workspace documentation:

```
apps/frontend/README.md
apps/backend/README.md
packages/contracts/README.md
```

While doing the remaining steps, record any instruction that is:

- unclear;
- missing;
- incorrect;
- in the wrong order;
- dependent on undocumented knowledge.

Step 4 - Install JavaScript dependencies

From the repository root, follow the documented dependency-installation procedure.

The expected clean-install command is:

```
npm ci
```

On Windows PowerShell, if necessary:

```
npm.cmd ci
```

Record:

- whether installation succeeded;
- whether any undocumented manual step was required;
- any warnings or errors that could affect onboarding.

Do not treat ordinary package-manager warnings as setup failures unless they prevent the documented workflow from working.

Step 5 - Configure the environment

Follow:

```
docs/environment.md
```

and the frontend/backend README instructions.

Create the required local environment files from the repository examples where documented.

Do **not** commit these files.

Confirm that the documentation makes it clear:

- which variables belong to the frontend;
- which variables belong to the backend;
- which Supabase values are required;
- which values are safe for the browser;
- which values must remain secret;
- which application functionality uses Supabase;
- that application-domain requests still use the handwritten backend API.

If a required environment variable is undocumented, record it as a verification failure.

Step 6 - Verify the shared contracts package

Follow:

```
packages/contracts/README.md
```

Verify that the documented contracts setup/build procedure works.

Where documented, run:

```
npm run build --workspace=@sport-analytics/contracts
```

or on Windows if required:

```
npm.cmd run build --workspace=@sport-analytics/contracts
```

Record whether this works without needing an undocumented setup step.

Step 7 - Run the repository quality checks

From the repository root, run the documented repository check:

```
npm run check
```

Windows PowerShell where required:

```
npm.cmd run check
```

Record whether it passes.

This check should exercise the repository's configured quality gates, including the applicable:

- repository structure validation;
- formatting;
- linting;
- contracts build;
- type checking;
- automated tests;
- API/OpenAPI validation where configured;
- production builds.

If it fails, copy the relevant error into the PR or verification record.

Do not fix an undocumented setup issue locally and simply report the test as passing.

Step 8 - Run the backend

Follow:

```
apps/backend/README.md
```

Start the backend using the documented command.

Expected root command:

```
npm run dev:backend
```

Windows PowerShell where required:

```
npm.cmd run dev:backend
```

Confirm that the backend starts successfully.

Where available, verify the health endpoint:

```
http://localhost:3000/api/v1/health
```

Record:

- whether startup succeeded;
- whether the documented port was correct;
- whether environment setup was sufficient;
- whether any undocumented command was needed.

Stop the backend after verification if necessary.

Step 9 - Run the frontend

In another terminal, follow:

```
apps/frontend/README.md
```

Start the frontend using the documented command.

Expected root command:

```
npm run dev:frontend
```

Windows PowerShell where required:

```
npm.cmd run dev:frontend
```

Open the documented local frontend address.

Expected default:

```
http://localhost:5173
```

Confirm that the application loads.

Record:

- whether startup succeeded;
- whether the documented port/address was correct;
- whether the frontend could read the required configuration;
- any undocumented steps that were required.

Step 10 - Verify MkDocs setup

Follow the MkDocs instructions in the developer documentation.

Run the strict documentation build:

```
python -m mkdocs build --strict
```

Record whether it succeeds.

The Material for MkDocs message concerning future MkDocs 2.0 compatibility may appear during the build. The current build should still complete successfully.

If the documentation describes serving the site locally, also follow that procedure and confirm that the documentation site opens correctly.

Step 11 - Review the technology-stack documentation

Read:

```
docs/development/technology-stack.md
```

Check whether the technologies you encounter while setting up the project are actually represented in the documentation.

In particular, verify that the documentation distinguishes between:

Frontend application	→ Azure App Service
Backend API	→ Azure App Service
Database	→ Supabase-hosted PostgreSQL
Authentication	→ Supabase Auth
Documentation website	→ Cloudflare Pages
Cloudflare deployment	→ Wrangler
Source control/workflow	→ Git + Gitea

If you encounter a direct project technology or development tool that is being used but is missing from the stack documentation, mention it.

Step 12 - Record your verification result

Update:

```
evidence/validation/issue-12-onboarding-verification.md
```

with your actual results.

Include at minimum:

- verifier name;
- date;
- operating system;
- editor/IDE;
- Git version;
- Node.js version;
- npm version;
- Python version;

- dependency installation result;
- contracts setup result;
- `npm run check` result;
- backend startup result;
- frontend startup result;
- MkDocs result;
- problems found;
- documentation changes required;
- final result: `PASS` or `FAIL / requires changes`.

Please do not mark something as passing if you needed an undocumented workaround.

Step 13 - Commit verifier evidence

If you update the verification record or fix documentation based on the test, commit those changes on this Pull Request branch according to the project Git methodology.

For a verification-only documentation commit, an example is:

```
docs(evidence): record independent onboarding verification
```

```
Refs #12
```

Only include an `Assisted-by:` trailer if AI actually assisted with that commit.

Step 14 - AI evidence if you use AI

If you use AI while performing the verification, diagnosing a failure, changing documentation or reviewing code, follow the repository AI policy.

You must:

- update **your own** AI usage register;
- save the relevant transcript/evidence where required;
- identify the actual tool and model used;

- record what you used it for;
- use the appropriate `Assisted-by:` commit trailer where required.

For example, only if you actually used that tool/model:

```
Assisted-by: ChatGPT-Web[GPT-5.6 Sol]
```

Do **not** copy another contributor's AI declaration if you did not use the same tool.

If your work introduces a new repository-wide AI tool or usage category, ensure the repository README/AI documentation remains accurate.

Step 15 - Report back on this PR

Please leave a PR comment summarising your test.

Include:

```
Clean-clone verification completed

OS:
Editor/IDE:
Node:
npm:
Python:

npm ci:
Contracts setup:
npm run check:
Backend startup:
Frontend startup:
MkDocs:

Documentation issues found:
Changes made:

Final result: PASS / REQUIRES CHANGES
```

If the result is `REQUIRES CHANGES`, keep this PR in Draft and fix the issues before the verification is repeated.

Additional Team Environment Information

Other team members do **not** all need to repeat the complete clean-clone exercise.

However, team members should add or comment with the following if their environment differs from what is already documented:

Name:
Operating system:
Editor/IDE:
Any project-specific setup difference:

This allows the onboarding documentation to reflect the development environments actually being used by the team without requiring six full clean-clone tests.

If adding this information requires a repository change, follow the normal branch/commit/AI evidence rules.

When This PR Can Leave Draft

This Pull Request may be marked ready for final review only when:

- ☐ at least one team member other than the author has completed the clean-clone procedure;
- ☐ the verification record contains the real results;
- ☐ the clean-clone result is **PASS** ;
- ☐ any documentation problems discovered during verification have been corrected and re-tested;
- ☐ the final installation and repository-check commands are confirmed;
- ☐ the dependency stated in Issue #12 regarding CI/reproducible installation has been satisfied;
- ☐ all relevant AI evidence has been recorded;
- ☐ the PR has received the review required by the team's Git methodology.

Related Issue

Refs #12

This Pull Request intentionally does **not** currently use `Closes #12`.

Issue #12 should only be automatically closed when all acceptance criteria are genuinely satisfied.

Immediately before final merge, if:

- independent verification has passed;
- all resulting fixes are complete;
- the CI/reproducible-installation dependency has been satisfied; and
- there is nothing else outstanding in Issue #12,

change:

`Refs #12`

to:

`Closes #12`

If any requirement is still incomplete, leave it as `Refs #12`.

Review Checklist

- ☐ All applicable Issue #12 acceptance criteria are met
- ☒ No unrelated product implementation is included
- ☒ Documentation and evidence have been updated
- ☒ Security implications were considered where relevant
- ☒ No credentials or secrets are intentionally committed
- ☒ Author AI assistance has been attributed
- ☐ Independent clean-clone verification has been completed
- ☐ Independent verification result is `PASS`
- ☐ Problems discovered during verification have been resolved
- ☐ The CI/reproducible-installation dependency has been satisfied
- ☐ At least one team member other than the author has reviewed the Pull Request

- ☐ Reviewer comments have been resolved
- ☐ Refs #12 has been changed to closes #12 only if Issue #12 can actually close

Evidence

Relevant evidence includes:

- evidence/validation/issue-12-onboarding-verification.md
- updated frontend, backend and contracts README files;
- updated developer setup and environment documentation;
- docs/development/technology-stack.md ;
- updated deployment documentation;
- updated MkDocs navigation;
- repository quality-check results;
- strict MkDocs build;
- AI usage register and transcript evidence;
- this Pull Request and its independent verification comments.

AI Declaration

The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Sol].

:::

This version makes the verification **very hard to fake accidentally**: they have to start clean, follow the docs, record exact versions/results, and report undocumented workarounds as failures rather than quietly solving them. That should give you strong evidence for the onboarding criterion.

8/13/2026, 4:18:26 PM



noo, cant we refer them to a doc or somethoing instead of putting it all in pr?

8/13/2026, 4:19:30 PM



Thinking



Inspecting Verification Document Content

- Inspected verification document content

✓ Worked for 20s

Yes — that is **much cleaner**. We already created exactly the right files for this.

The teammate should use:

- **Actual step-by-step onboarding guide:** `docs/development/setup.md`
- Supporting setup docs:
 - `apps/frontend/README.md`
 - `apps/backend/README.md`
 - `packages/contracts/README.md`
 - `docs/environment.md`
- **Verification checklist/evidence form:** `evidence/validation/issue-12-onboarding-verification.md`

So the PR should **not duplicate 15 steps**. It should tell them to follow the documentation as if they were a new developer, then complete the verification record.

Replace that huge verification section in the PR with this:

```
## Independent Clean-Clone Verification Required
```

```
**This Pull Request must remain a Draft until the independent onboarding verification has been completed.**
```

A team member other than the author must perform a genuine clean-clone test of this branch.

What the verifier must do

1. Clone the repository into a completely new directory and check out this PR branch.
2. Follow the developer onboarding instructions in:

- ``docs/development/setup.md``
- ``apps/frontend/README.md``
- ``apps/backend/README.md``
- ``packages/contracts/README.md``
- ``docs/environment.md``

3. Do not rely on an existing ``node_modules``, ``.env`` files, build output or previously configured checkout.
4. Record the actual verification results in:

``evidence/validation/issue-12-onboarding-verification.md``

5. Record any:
 - unclear instructions;
 - missing steps;
 - failed commands;
 - undocumented workarounds;
 - incorrect environment information; or
 - technologies/tools encountered that are missing from the documentation.
6. If problems are found, update the documentation on this branch and repeat the affected verification steps before recording a final ``PASS``.
7. Leave a short PR comment confirming the result once complete.

Important

Please do not silently work around an onboarding problem.

If the documented instructions do not work from a clean clone, record the problem so that the guide can be corrected.

If you make changes while testing, follow the normal Git methodology.

If you use AI while testing, diagnosing problems or making changes:

- update your own AI usage register;
- save the required transcript/evidence;
- identify the actual tool and model used; and
- include the appropriate `Assisted-by:` trailer where required.

Do not copy another contributor's AI declaration if you did not use that tool/model.

Then your **When This PR Can Leave Draft** section can stay short:

`## When This PR Can Leave Draft`

This Pull Request may be marked ready for final review when:

- [] an independent team member has completed the clean-clone verification;
- [] `evidence/validation/issue-12-onboarding-verification.md` contains the real results;
- [] the final verification result is `PASS`;
- [] any problems discovered during verification have been corrected;
- [] the CI/reproducible-installation dependency stated in Issue #12 has been satisfied;
- [] relevant AI evidence has been recorded; and
- [] normal PR review requirements have been met.

And retain:

`## Related Issue`

Refs #12

with your note:

Change this to `closes #12` only before final merge if every acceptance criterion and dependency is actually complete.

This is better because **the PR tells them what evidence is required, while the repo itself proves that your onboarding documentation is usable**. If the PR contained all the setup instructions, they could successfully set up from the PR while your actual developer guide was still incomplete—which would defeat the purpose of #12.

8/13/2026, 4:19:32 PM